

→ LR(1) ^{Items} ~~Version~~ : ~

LR(1) is an improved version of SLR(1). The main difference is how it decides when to reduce.

→ What does LR(1) mean?

- L - Scan input from left to Right
- R - Construct Rightmost derivation in reverse
- 1 - Use 1 lookahead symbol

→ Why do we need LR(1)?

→ LR(0) :

if we have :
 $A \rightarrow \alpha$.

LR(0) says :

Reduce by $A \rightarrow \alpha$ for all terminals.

This can create conflicts.

→ SLR(1) :

SLR improves this :

$A \rightarrow \alpha$.

Reduced when :

lookahead $\in \text{Follow}(A)$

lookahead $\in \text{Follow}(A)$

But $\text{Follow}(A)$ can still be too broad

→ $\text{LR}(1)$:

$\text{LR}(1)$ goes one step further.

It stores the exact lookahead with the item.

Eg ;

$[A \rightarrow \alpha \cdot, b]$

means

Reduce $A \rightarrow \alpha$ only when the next symbol is b .

This makes $\text{LR}(1)$ more precise than $\text{SLR}(1)$

→ $\text{LR}(1)$ Item :

An $\text{LR}(1)$ item looks like :

$A \rightarrow \alpha \cdot \beta$

An $\text{LR}(1)$ item adds a lookahead :

Page No. _____
Date ____/____/____

$$[A \rightarrow \alpha \cdot \beta, a]$$

where :

- $A \rightarrow \alpha \cdot \beta$ = LR(0) item
- a = lookahead symbol

eg; ①

$$[E \rightarrow E + \cdot T, \$]$$

means:

- We have already recognized $E +$
- we expect T
- $\$$ is the lookahead

eg ②

$$[C \rightarrow d \cdot, c]$$

means:

we can reduce $C \rightarrow d$ when the next ~~symbol~~ input symbol is c .

→ Most important difference:

Suppose we have:

$C \rightarrow d.$

LR(0); reduction on:

$C, d, \$$

or generally every terminal.

SLR(1):

if:

$FOLLOW(C) = \{C, d, \$\}$

then reduction on:

$C, d, \$$

LR(1);

Suppose the item is:

$[C \rightarrow d, c]$

Then reduction happens only on c .

$c \quad d \quad \$$
R3

This is the key idea.

→ Form of LR(1) item:

The general form is:

$$[A \rightarrow \alpha \cdot \beta, a]$$

There are two important cases:

Case 1: Dot is before a symbol.

$$[A \rightarrow \alpha \cdot \beta, a]$$

This indicates that we will have something to recognize.

We may perform SHIFT or GOTO.

Case 2: Dot is at the end:

$$[A \rightarrow \alpha \cdot, a]$$

This means the production is complete.

Now:

Reduce by $A \rightarrow \alpha$ only when the next input symbol is a .

→ LR(1) Closure:

This is where LR(1) become more interesting than LR(0).

Suppose we have:

~~[A → α. Bβ, a]~~

$[A \rightarrow \alpha. B\beta, a]$

The dot is before non-terminal B.

we must add production of B.

But we must calculate their lookahead using:

~~First~~ FIRST(βa)

So:

$[B \rightarrow \cdot Y, b]$

is added for every:

$$b \in \text{FIRST}(\beta a)$$

This is the important rule of LR(1) closure.

Eg;

for LR(1) closure:

$$S \rightarrow CC$$

$$C \rightarrow cC$$

$$C \rightarrow d$$

⇒

Augmented grammar:

$$S' \rightarrow S$$

$$S \rightarrow CC$$

$$C \rightarrow cC$$

$$C \rightarrow d$$

Start item:

$$[S' \rightarrow \cdot S, \$]$$

Since dot is before S, add:

~~$[S \rightarrow \cdot cc, \$]$~~
~~Not done dot before CT~~
 ~~$C \rightarrow \cdot cc, \$$~~

first find $FIRST(\cdot \$)$

$= \$$

$\therefore$

$[S \rightarrow \cdot cc, \$]$

Now, dot is before c, add:

first find $FIRST(c\$)$

$= FIRST(c) = \{c, d\}$

$\therefore$

$[C \rightarrow cC, c] \Rightarrow [C \rightarrow cC, c/d]$

$[C \rightarrow cC, d]$

$[C \rightarrow d, c] \Rightarrow [C \rightarrow d, c/d]$

$[C \rightarrow d, d]$

∴

I₀:

$[s' \rightarrow \cdot s, \$]$

$[s \rightarrow \cdot cc, \$]$

$[C \rightarrow \cdot cc, c/d]$

$[C \rightarrow \cdot d, c/d]$

I₁:

$[s' \rightarrow s, \$]$

This is ACCEPT State.

Action $[1, \$] = \text{Accept}$

I₂:

$[s \rightarrow C \cdot c, \$]$

$[C \rightarrow \cdot cc, \$]$

$[C \rightarrow \cdot d, \$]$

$I_3:$

$[C \rightarrow c.c, c/d]$

$[C \rightarrow .cc, c/d]$

$[C \rightarrow .d, c/d]$

$I_4:$

$[C \rightarrow d., c/d]$

$I_5:$

~~$[S \rightarrow cc., \$]$~~

$I_6:$

$[C \rightarrow c.c, \$]$

$[C \rightarrow .cc, \$]$

$[C \rightarrow .d, \$]$

$I_7:$

$[C \rightarrow d., \$]$

$I_8:$

~~$[C \rightarrow cc., c/d]$~~

Iq :

$[C \rightarrow c C, \$]$

GOTO Transition :-

$I_0 \rightarrow S \rightarrow I_1$

$I_0 \rightarrow C \rightarrow I_2$

$I_0 \rightarrow c \rightarrow I_3$

$I_0 \rightarrow d \rightarrow I_4$

~~I_1~~

$I_2 \rightarrow C \rightarrow I_5$

$I_2 \rightarrow c \rightarrow I_6$

$I_2 \rightarrow d \rightarrow I_7$

$I_3 \rightarrow C \rightarrow I_8$

$I_3 \rightarrow c \rightarrow I_3$

$I_3 \rightarrow d \rightarrow I_4$

$I_6 \rightarrow C \rightarrow I_9$

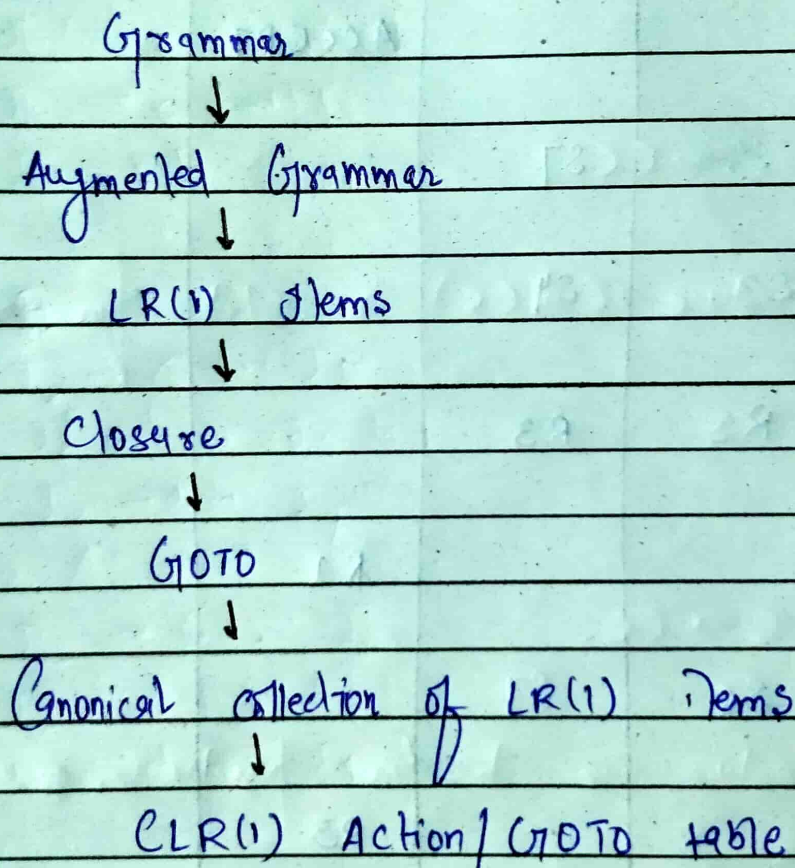
$I_6 \rightarrow c \rightarrow I_6$

$I_6 \rightarrow d \rightarrow I_7$

→ $CLR(1) :-$

$CLR(1)$ means Canonical $LR(1)$. It is the full $LR(1)$ parsing method we were constructing above.

→ Steps of ~~LR~~ $CLR(1) :-$



Note :-

Shift Entries & GOTO Entries same as we before done in $LR(0)$, $SLR(1)$

⇒ We take the previous example that we solve and find state and goto transition.

CLR(1) Parsing Table :

state	Action			GOTO	
	c	d	\$	S	C
0	S3	S4		1	2
1			ACCEPT		
2	S6	S7			5
3	S3	S4			8
4	R3	R3			
5			R1		
6	S6	S7			9
7			R3		
8	R2	R2			
9			R2		

As we know,

Shift and Goto Entries same as in LR(0).

$\therefore$ we can fill table of CLR(1) from the given GOTO transition.

Till State I_3 No new thing we have to do.

from I_4 :

$C \rightarrow d, c/d$

Reduced by

$C \rightarrow d$

[Production 3]

for c/d means for both c & d .

I_5 :

$S \rightarrow CC, \$$

Reduced in $\$$ Production 1

$T_7:$

$C - d, \$$

reduced in $\$$ for production 3

$T_8:$

$C \rightarrow CC.$

Reduce $(C \rightarrow CC)$ in C/d for production 2

$T_9:$

$C \rightarrow CC.$

Reduce in $\$$ for production 2

Hence fill the CLR(1) parsing table.

Aakash